

MACHINE LEARNING — LAB MANUAL

Lab 5: Linear Regression — Predicting House Prices

A 3-Hour Hands-On Lab — BS Computer Science, 5th Semester

Duration: 3 Hours (independent, hands-on)

Prerequisites: Labs 1–4 (Python/tools, cleaning/EDA, Decision Trees, Random Forest/CV/tuning); Linear Regression theory (simple & multiple, coefficients, residuals, MSE/RMSE/MAE/R²) from the AI/ML course

Tools Required: Python 3.x, Pandas, NumPy, scikit-learn, Matplotlib, Google Colab

Includes: A complete regression workflow, coefficient interpretation, residual analysis, MAE/MSE/RMSE/R², overfitting check, and a graded rubric

Learning Objectives

This lab is NOT a re-introduction to Linear Regression theory — you've already studied intercepts, coefficients, residuals, and regression metrics. The goal is to practice implementing, evaluating, and interpreting a complete regression workflow on a new dataset and a new problem type (regression, not classification). By the end of this lab you should be able to independently:

- Identify the dependent variable and numerical/categorical independent variables in a new dataset
- Prepare mixed numerical/categorical data for scikit-learn's LinearRegression
- Train a Linear Regression model and generate predictions
- Extract and interpret the intercept and coefficients in the context of house prices
- Analyze residuals visually and diagnose systematic patterns
- Evaluate a regression model using MAE, MSE, RMSE, and R²
- Compare training vs. test performance to assess generalization
- Judge which features are most influential and discuss the model's limitations

Module	Focus Area
1	Dataset, Problem & Feature/Target Identification (0–20 min)
2	Data Preparation & Train/Test Split (20–45 min)
3	Linear Regression & Predictions (45–75 min)
4	Coefficients & Intercept (75–105 min)
5	Residuals & Regression Metrics (105–135 min)

Module	Focus Area
6	Training vs. Testing & Generalization (135–160 min)
7	Final Model Analysis & Conclusion (160–180 min)

Dataset & Problem

You will work with the "[House Prices - Advanced Regression Techniques](#)" dataset from Kaggle — a real, ongoing Kaggle competition. It contains 79 explanatory features (a mix of numerical features like living area, lot size, and year built, and categorical features like neighborhood, house style, and quality ratings) describing residential homes in Ames, Iowa, with a continuous target: SalePrice. This is a different, larger dataset than the one used in your Linear Regression theory examples — the goal here is to apply what you already know to a genuinely new problem, not repeat a worked example.

Item	Detail
Competition name	House Prices — Advanced Regression Techniques
Official Kaggle link	kaggle.com/competitions/house-prices-advanced-regression-techniques
Files you need	train.csv (1,460 labeled houses), test.csv (1,459 unlabeled houses), sample_submission.csv
Target variable	SalePrice (train.csv only — test.csv has no SalePrice column)
Kaggle's own evaluation metric	RMSE between log(predicted price) and log(actual price)

Before You Start

- Create a free Kaggle account if you don't have one, and join the competition at the link above (click "Join Competition" / accept the rules — required before you can submit).
- Download train.csv, test.csv, and sample_submission.csv from the competition's "Data" tab and upload them to your Colab session (or mount Google Drive).
- This dataset has 79 raw features — full feature engineering across all of them is out of scope for this lab (that belongs to a later course). Instead, select a manageable subset of roughly 10–12 features, mixing numerical and categorical, that you'd reasonably expect to matter for price (e.g. living area, overall quality, total basement area, garage capacity, year built, bathrooms, neighborhood, house style, kitchen quality).
- Create a new Colab notebook: lab5_<yourname>_house_price.ipynb, with the student-info block and Markdown headings described in Submission Requirements.

Part 1 — Dataset, Problem & Feature/Target Identification

Before writing any modeling code, understand what you're predicting and what you're predicting it from — this is a regression problem, not classification, so revisit what that distinction means for evaluation later in the lab.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
pd.read_csv(...)	Load the house-price dataset
df.shape / df.info() / df.head()	Inspect rows, columns, and dtypes
df.describe()	Summary statistics for numerical columns

Tasks

1. Load the dataset and briefly inspect it (shape, dtypes, first few rows, summary statistics).
2. Identify the dependent variable and list the independent variables, split explicitly into numerical features and categorical features.
3. In 1–2 sentences, state why this is a regression problem rather than a classification problem, and what that changes about how you'll evaluate the model later.

Part 2 — Data Preparation & Train/Test Split

scikit-learn's LinearRegression requires fully numeric input, just like the classifiers in Labs 3–4. Convert categorical columns appropriately, then hold out a test set before training anything.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>df[col].map({'yes': 1, 'no': 0})</code>	Encode a binary Yes/No column
<code>pd.get_dummies(df, columns=[...])</code>	One-hot encode a multi-category column
<code>train_test_split(X, y, test_size=..., random_state=...)</code>	Splits data into training and test sets

Tasks

4. Prepare X (features) and y (price) for modeling: encode every categorical column into numeric form, and briefly justify your choice of encoding method for each type of column.
5. Confirm X is fully numeric with no missing values.
6. Split X and y into training and test sets (e.g. an 80/20 split) and report the resulting shapes.

Part 3 — Linear Regression & Predictions

Train a Multiple Linear Regression model using all your prepared features, and generate price predictions on the test set.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>from sklearn.linear_model import LinearRegression</code>	Import the regression model
<code>LinearRegression().fit(X_train, y_train)</code>	Trains the model
<code>.predict(X)</code>	Generates predicted prices for a given feature set

Tasks

7. Train a LinearRegression model on the training data.
8. Generate predicted prices on the test set, and display a small table comparing actual vs. predicted price for 10 test houses.

Part 4 — Coefficients & Intercept

The intercept and coefficients are where a Linear Regression model's "reasoning" becomes visible — unlike a Decision Tree or Random Forest, you can read the exact effect of each feature directly off the model.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>model.intercept_</code>	The model's b_0 (predicted price when all features are 0)
<code>model.coef_</code>	The model's $b_1...b_n$ (one coefficient per feature)
<code>pd.DataFrame({'Feature': X.columns, 'Coefficient': model.coef_})</code>	Builds a readable coefficient table

Tasks

- Extract the intercept and all coefficients. Build a table: Feature | Coefficient | Interpretation, with a one-line, plain-English interpretation for each feature (e.g. "each additional bathroom is associated with a price increase of approximately ___, holding other features constant").
- Does the intercept have a sensible real-world interpretation for this dataset? Why or why not?

Part 5 — Residuals & Regression Metrics

Residuals (actual – predicted) reveal patterns a single accuracy number can't. Analyze them visually, then evaluate the model with the standard regression metrics.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>residuals = y_test - y_pred</code>	Computes the residual for each prediction
<code>plt.scatter(y_pred, residuals)</code>	Residuals vs. predicted values plot
<code>plt.hist(residuals, bins=...)</code>	Distribution of residuals
<code>mean_absolute_error, mean_squared_error, r2_score</code>	Standard regression metrics (compute RMSE as sqrt of MSE)

Tasks

- Compute residuals on the test set and plot them (e.g. residuals vs. predicted values, and a histogram of residuals). Do the residuals look randomly scattered, or is there a visible pattern?
- Compute MAE, MSE, RMSE, and R^2 on the test set. Report them in a small table.
- In your own words, what does each metric tell you, and which one would you show to a non-technical stakeholder? Why?

Part 6 — Training vs. Testing & Generalization

Just as in Lab 3's overfitting experiment, compare performance on data the model was trained on vs. data it has never seen.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>model.predict(X_train)</code>	Predictions on the training set, for comparison
<code>r2_score(y_train, train_pred)</code> vs. <code>r2_score(y_test, test_pred)</code>	Compare R^2 across train and test

Tasks

14. Compute MAE, MSE, RMSE, and R^2 on the training set, and place them side by side with your Part 5 test-set metrics in one comparison table.
15. Is there a large gap between training and testing performance? What would a large gap suggest about overfitting or generalization, and what does your actual result suggest here?

Part 7 — Final Model Analysis, Conclusion & Kaggle Submission

Step back from individual metrics and decide, with evidence, what you've actually learned about predicting house prices with Linear Regression — then experience the full Kaggle workflow by submitting your predictions and checking your leaderboard rank.

Tasks

16. Looking back at your Part 4 coefficient table, which 2–3 features appear most influential on price? Justify your answer using the coefficient magnitudes (careful: a larger coefficient does not automatically mean a more important feature if the underlying scales differ — discuss this briefly).
17. Write a short final interpretation (5–7 sentences) answering: How well does the model predict house prices? Which metric is most useful and why? Which feature has the strongest effect? What limitations does Linear Regression have for this dataset? What would you try next?
18. Apply the same feature selection and encoding you used on `train.csv` to `test.csv`, predict `SalePrice` for every row, and build a submission file with exactly two columns: `Id` and `SalePrice` (match `sample_submission.csv`'s format). Save it as `submission.csv`.
19. Submit `submission.csv` on the competition's "Submit Predictions" page. Record your public leaderboard score and rank, and write 1–2 sentences on how it felt to see your model measured against thousands of other real submissions.

Submission Requirements

Submit exactly ONE Google Colab notebook. At the very top, include a student-info block with: Name, Student ID, Section, GitHub, Kaggle, Dataset, and Dataset Source.

Your Notebook Must Contain These Sections

- 1. Problem Definition 2. Dataset Exploration 3. Feature and Target Identification 4. Data Preparation
- 5. Train/Test Split 6. Linear Regression Model 7. Predictions 8. Coefficients and Intercept

- 9. Residual Analysis 10. Regression Metrics 11. Training vs Testing Performance 12. Model Interpretation 13. Conclusion

Every table, plot, and metric must be accompanied by a short written interpretation — code with no discussion will not be accepted.

Kaggle Leaderboard Submission (Part 7)

- Submit your submission.csv on the competition page and record your public leaderboard score and rank in your notebook's Conclusion.
- Your rank itself is not graded — everyone's feature subset and model will differ — but a missing or invalid submission.csv (wrong column names, wrong row count) will lose marks.